Implementation Take-Home: Multi-Tenant Record Ingestion

Build a working prototype that accepts records from multiple customers, validates them against versioned schemas, and stores valid records.

Timebox: Aim to spend no more than eight hours. Focus on correctness, clear behavior, and a reproducible demonstration. Include a brief note on what you completed and any areas you would explore with more time.

AI assistance is expected. Include your source code and the AI session transcripts used for this assignment. Use AI throughout the process, and be prepared to walk us through your approach and how you checked the results.

Background and scope

Customers submit records in batches over HTTP. Each record belongs to a dataset with a customer-defined schema that can evolve over time. Your service sits between incoming batches and downstream storage.

Production context is 50,000 records per second across hundreds of customers. **Your prototype does not need to achieve that throughput.** Demonstrate the core behavior locally and explain how your design could evolve to meet that scale.

Choose your language, framework, datastore, and API design. A single process and local storage are sufficient; no hosted account should be needed. Endpoint paths, request formats, scheduling, and storage design are yours to choose and document.

1. Versioned schemas

Customers register schemas for their datasets. Support flat fields of type **string, integer, float, boolean, and timestamp**, each marked required or optional. Schemas must remain independent across customers, even when dataset names match.

- The initial schema may contain required fields. Compare subsequent versions with the latest registered version; additions must be optional. Reject field removal, type changes, new required fields, and changes to an existing field's required/optional setting.
- Registered versions are immutable, and older versions remain available for validation.
- Repeating an identical registration must be safe. Reject conflicting content for an existing version.

Reject malformed schemas, such as duplicate field names or unsupported types, without modifying registered schemas. For example, adding an optional **note** field is allowed; changing **amount** from float to string is not.

2. Batch submission

Each batch belongs to one customer and dataset and has a client-supplied batch identifier. Each record has an identifier, schema version, and payload. Records in the same batch may use different schema versions. An illustrative request:

```
{
  "customer_id": "customer_123", "dataset_id": "orders",
  "batch_id": "batch_001",
  "records": [{
    "record_id": "record_001", "schema_version": 1,
    "payload": {"order_id": "order_001", "amount": 42.5}
  }]
}
```

Submission must not wait for record processing to finish. Expose its processing status and final report. Choose and document request-size and batch-size limits.

3. Validation, storage, and reporting

Validate each record against its explicitly requested schema version. Store valid records even when other records in the batch fail.

- Reject missing required fields, explicit null values, undeclared fields, and incorrect types. Optional fields may be omitted.
- Do not coerce values between types; booleans are not numbers. Support flat payloads only. Document your numeric and timestamp validation rules.
- An unknown dataset or schema version, or a malformed record, fails that record.
- Reject malformed JSON, invalid batch-level structure, empty batches, duplicate valid record identifiers, and oversized submissions before processing any records. Malformed individual records fail independently.

For this prototype, downstream storage is simply the persistent store holding successful records; it may share a datastore with schemas and batch metadata. Provide an API, command, or documented query to inspect the stored records.

During uninterrupted operation, every accepted batch must eventually produce a completed report with:

- Customer, dataset, and batch identifiers.
- Total, successful, and failed record counts that reconcile; every record is counted exactly once.
- One entry per failed record with a clear reason, distinguishing validation from storage failures. Identify it by record identifier, or batch position if its identifier is missing or invalid.

A batch containing only invalid records must still complete with a report. Mark a batch completed only after every record has a final outcome and every successful record is available in storage. Completed reports must remain unchanged. Schemas, stored records, and completed reports must survive a normal restart.

4. Safe retries and failures

A client may retry a submission, including while the original is processing. Retrying the same batch must return its existing status or result without starting additional processing or creating duplicate stored records. The original processing may continue. Reusing a batch identifier with different content must be rejected.

Batch identifiers are scoped to a customer and dataset. Deduplication across different batches is not required. Choose and document how you identify equivalent submissions and prevent duplicate writes.

Automatically retry transient storage failures using a bounded policy. After retries are exhausted, report the affected record as failed and continue processing other records. Do not retry validation failures. Make delays and write failures reproducible; a test-only storage wrapper is sufficient. Required fault injection may fail writes before persistence. Discuss uncertain outcomes after a write commits in your design notes; implementing that handling is not required.

5. Customer isolation

One customer's large or slow batch must not monopolize processing capacity; other customers must continue making progress. Choose a scheduling and concurrency approach that uses bounded resources.

Demonstrate this by starting a large batch for customer A, deliberately slowing A's storage writes, and then submitting a small batch for customer B. Show B completing while A is still processing. Report batch sizes, injected delay, and completion times.

There is no fixed latency target. Explain the isolation your implementation provides and its limitations.

Required verification

Use small, deterministic fixtures and focused integration tests through the public API. Hand-authored expected results are sufficient; do not reuse service validation logic to calculate them. No separate checker framework or large fixture generator is required. Verify successful records appear exactly once with unchanged payloads, and invalid records are absent.

Scenario	Expected outcome
Schema evolution and older versions	Allowed changes succeed; malformed schemas and prohibited changes leave schemas unchanged; requested versions are used.
Mixed, all-invalid, and malformed records	Only valid records are stored; every record is accounted for with accurate failures.
Repeated or conflicting submissions	Retries, including during processing, create no duplicates; conflicting content is rejected.
Transient and exhausted storage failures	Transient failures recover without duplicates; exhausted failures are reported while other records continue.
Customers sharing a dataset name	Schemas and stored records remain separate.
Large or slow batch alongside a small batch	The other customer's small batch finishes while the slow batch is still processing.
Normal restart after all batches complete	Registered schemas, stored records, and completed reports remain available and unchanged.

Out of scope

No UI, cloud deployment, authentication, real message broker, multi-instance deployment, production throughput benchmark, cross-batch deduplication, or global ordering is required. Treat customer identifiers as trusted input.

Manual reprocessing and recovery of in-flight batches after a process crash are also out of scope. Explain production crash recovery in your design notes; you do not need to implement it.

Deliverables

Source code and README. Include prerequisites, exact commands to start dependencies and the service and run tests/demonstrations, API examples, fixtures, failure-injection support, and a way to inspect stored records.

DECISIONS.md (1-2 pages). Describe the architecture and key tradeoffs, including schema lookups, validation semantics, retries, duplicate prevention, and customer isolation. Describe current limitations and what would change for production throughput, horizontal scaling, and crash recovery. Include how you checked the results and any AI suggestions you adapted or chose not to use.

AI session transcripts. Preserve the interaction history used for the assignment. Redact credentials or sensitive information and mark redactions. The transcripts help us understand how you guided the work and checked the results.

Include your approximate time spent and any unfinished areas. Where behavior is left open, document your choice and reasoning.

How we evaluate

We evaluate correctness, reliability, customer isolation, verification, and engineering judgment. Prioritization and tradeoffs matter alongside completeness. Focus on core behavior over extra features.